Как настроить и использовать clang-format и clang-tidy

Файл или инструмент	Что делает	Чего от него ожидать
.clang-format	Хранит правила оформления	Отступы, пробелы, скобки, переносы
clang-format	Применяет .clang-format	Может изменить оформление файлов
.clang-tidy	Хранит правила анализа	Имена и выбранные проверки C++
clang-tidy	Анализирует код с учетом компиляции	Сообщает о нарушениях; требует заголовки C++
check_style.py	Запускает готовую проверку проекта	Режим проверки либо форматирование
tools-requirements.txt	Фиксирует версии инструментов	У всех участников одинаковые версии

1. Что такое эти файлы

1.1. .clang-format — настройки оформления

Это текстовый файл с точным именем `.clang-format`: точка в начале, расширения .txt нет. Он сам ничего не запускает. Программа clang-format читает его, когда обрабатывает исходник. Настройки ищутся рядом с исходным файлом, затем в родительских каталогах. Для нашего проекта достаточно одного файла в корне; вложенный .clang-format может изменить правила для своего каталога, поэтому случайные дополнительные конфигурации не нужны.

1.2. clang-format — программа форматирования

Она расставляет отступы, пробелы, скобки и переносы. Не переименовывает переменные и методы, не проверяет смысл имен и не исправляет алгоритм. С `-i` изменяет файлы на месте. С `--dry-run --Werror` только проверяет форматирование и возвращает ошибку при несоответствии. Без `-i` выводит отформатированный текст в терминал, оставляя исходник прежним.

1.3. .clang-tidy — настройки анализа

Это отдельный текстовый файл, также без .txt. Он задает список проверок и параметры именования. В комплекте включены конкретные проверки, а не все существующие. `WarningsAsErrors: '*'` делает найденные предупреждения выбранных проверок причиной неуспешного результата; при этом проверки, отключенные в Checks, не включаются.

1.4. clang-tidy — анализатор C++

Он должен понимать код почти как компилятор: знать стандарт языка, каталоги заголовков и параметры сборки. Установка одного clang-tidy не устанавливает полный компилятор, стандартную библиотеку и системный SDK. Заголовки, подключенные из .cpp, также анализируются; передавать .h отдельно для этой проверки не нужно. По умолчанию анализ не переписывает исходники.

1.5. check_style.py — готовый запуск

Скрипт обрабатывает собственные C++-файлы в include и src. Обычный запуск проверяет форматирование и clang-tidy, не изменяя код. `--format` только форматирует файлы; после него требуется обычная проверка. Скрипт проверяет точные версии инструментов и завершает работу с ненулевым кодом, если проверка не прошла. Он рассчитан на структуру примера; для другого расположения исходников список каталогов нужно изменить.

1.6. tools-requirements.txt — одинаковые версии

В комплекте закреплены clang-format 23.1.3 и clang-tidy 22.1.8. Версии различаются, поскольку программы выполняют независимые задачи. Используются пакеты PyPI, которые распространяют готовые инструменты LLVM; это способ установки, а не требование языка C++. Если подходящего готового пакета для вашей архитектуры нет, используйте поддерживаемый компьютер либо согласуйте установку совместимой сборки инструментов. Самовольно обновлять версии в середине курса не следует: форматирование и проверки могут измениться.

2. С чего начать

Распакуйте `cpp_course_style` в удобный каталог. Корень проекта — папка, где лежат .clang-format, .clang-tidy, tools-requirements.txt и check_style.py. Именно из нее выполняются команды ниже. Исходники, а не один открытый файл, должны находиться внутри этой папки. В Windows включите показ расширений файлов и убедитесь, что конфигурации не названы `.clang-format.txt`.

Чтобы перейти в проект, выберите только блок своей системы и замените пример пути на ваш путь. PowerShell и CMD — разные оболочки Windows: команду CMD не нужно вставлять в PowerShell и наоборот. Linux ниже означает Debian/Ubuntu; для другого дистрибутива команды установки пакетов отличаются. Команды терминала macOS выполняются в обычном Terminal.

PowerShell

```
Set-Location -LiteralPath 'C:\cpp_course_style'
```

CMD

```
cd /d "C:\cpp_course_style"
```

Linux

```
cd "$HOME/cpp_course_style"
```

macOS

```
cd "$HOME/cpp_course_style"
```

Угловые обозначения приглашения терминала вроде `PS>` или `$` в команды не добавляются. Каждая строка выполняется отдельно. Если установка или создание окружения завершились с ошибкой, сначала устраните ее и только затем переходите к следующим шагам.

3. Установка: один раз для проекта

3.1. Windows: Python и поддержка C++

Если Python уже установлен и `py -3 --version` работает, повторная установка не нужна. Иначе установите Python через официальный установщик с python.org либо через winget. После установки закройте и снова откройте терминал, чтобы он получил обновленные пути.

PowerShell:

```
winget install --exact --id Python.Python.3.13
```

CMD:

```
winget install --exact --id Python.Python.3.13
```

Если winget не найден, используйте официальный установщик Python. Для анализа и сборки C++ установите Visual Studio Community или отдельные Build Tools с компонентом «Разработка классических приложений на C++» / Desktop development with C++. Выберите MSVC и Windows SDK. Установленный редактор VS Code сам по себе эти компоненты не предоставляет. Если C++ уже собирается в Visual Studio, обычно компоненты установлены.

Для анализа откройте из меню Пуск Developer PowerShell for VS либо x64 Native Tools Command Prompt for VS. Первый использует команды PowerShell, второй — CMD. Обычного терминала достаточно для форматирования. Сборка и clang-tidy с MSVC выполняются в подготовленном терминале разработчика. После его открытия снова перейдите в корень проекта. Для нестандартной установки MSVC предпочитайте ярлык разработчика, а не ручное копирование INCLUDE и LIB с чужого компьютера.

3.2. Linux: Debian / Ubuntu

Установите Python, поддержку виртуального окружения и C++-компилятор. sudo может запросить пароль пользователя.

```
sudo apt update
sudo apt install python3 python3-venv python3-pip g++
```

Другие дистрибутивы используют свой менеджер пакетов; нужны Python 3, venv и установленная стандартная библиотека C++. Для этого небольшого проекта анализ использует заголовки компилятора системы.

3.3. macOS

Установите инструменты разработчика Apple; команда открывает окно установки. Дождитесь ее завершения.

```
xcode-select --install
```

Для Python можно использовать официальный установщик python.org. Если вы выбираете Homebrew и его еще нет, официальный установщик запускается так; после выполнения следуйте напечатанным им Next steps для добавления brew в PATH:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Если Homebrew установлен, первая команда не нужна. Установка Python через него:

```
brew install python
```

3.4. Создание окружения и установка инструментов

В корне проекта создается `.style-venv`. Это локальная папка с Python и инструментами данного проекта. Она не отправляется преподавателю и не копируется между разными ОС. Активация окружения не требуется: ниже используются явные пути. Поэтому изменение ExecutionPolicy в PowerShell также не требуется.

PowerShell:

```
py -3 -m venv .style-venv
.style-venv\Scripts\python.exe -m pip install -r tools-requirements.txt
```

CMD:

```
py -3 -m venv .style-venv
.style-venv\Scripts\python.exe -m pip install -r tools-requirements.txt
```

Linux:

```
python3 -m venv .style-venv
.style-venv/bin/python -m pip install -r tools-requirements.txt
```

macOS:

```
python3 -m venv .style-venv
.style-venv/bin/python -m pip install -r tools-requirements.txt
```

Обе операции требуют Python; установка инструментов требует доступа к интернету. Если `.style-venv` уже создано, заново создавать его не нужно: выполните установку требований существующим Python из него.

3.5. Проверка версий

PowerShell:

```
.style-venv\Scripts\clang-format.exe --version
.style-venv\Scripts\clang-tidy.exe --version
```

CMD:

```
.style-venv\Scripts\clang-format.exe --version
.style-venv\Scripts\clang-tidy.exe --version
```

Linux:

```
.style-venv/bin/clang-format --version
.style-venv/bin/clang-tidy --version
```

macOS:

```
.style-venv/bin/clang-format --version
.style-venv/bin/clang-tidy --version
```

В выводе должны быть 23.1.3 у clang-format и 22.1.8 у clang-tidy. Текст вокруг номера может различаться. Программа, случайно найденная в PATH, может иметь другую версию; явные пути выше устраняют эту неоднозначность.

4. Ежедневная работа: две команды

Сначала сохраните файлы и примените форматирование. Эта команда изменяет C++-исходники в include и src:

PowerShell:

```
.style-venv\Scripts\python.exe check_style.py --format
```

CMD:

```
.style-venv\Scripts\python.exe check_style.py --format
```

Linux:

```
.style-venv/bin/python check_style.py --format
```

macOS:

```
.style-venv/bin/python check_style.py --format
```

Затем запустите проверку оформления и выбранных правил C++. На Windows используйте терминал разработчика, описанный выше:

PowerShell:

```
.style-venv\Scripts\python.exe check_style.py
```

CMD:

```
.style-venv\Scripts\python.exe check_style.py
```

Linux:

```
.style-venv/bin/python check_style.py
```

macOS:

```
.style-venv/bin/python check_style.py
```

Если все прошло, появится `Formatting and clang-tidy checks passed.`. Если есть ошибка, исправьте указанный код и запустите проверку снова. После успешной проверки соберите программу и проверьте ее поведение: анализ стиля не заменяет запуск.

5. Ручной запуск: как работают параметры

5.1. Отформатировать один файл

PowerShell:

```
.style-venv\Scripts\clang-format.exe -i src\student_report.cpp
```

CMD:

```
.style-venv\Scripts\clang-format.exe -i src\student_report.cpp
```

Linux:

```
.style-venv/bin/clang-format -i src/student_report.cpp
```

macOS:

```
.style-venv/bin/clang-format -i src/student_report.cpp
```

`-i` означает изменение исходного файла. После команды откройте его и посмотрите результат. Никакой отдельной команды «подключить .clang-format» не нужно: файл автоматически найден в родительском каталоге. Для явного указания можно использовать `--style=file:.clang-format`, но обычно это избыточно.

5.2. Проверить оформление без изменения

PowerShell:

```
.style-venv\Scripts\clang-format.exe --dry-run --Werror src\student_report.cpp
```

CMD:

```
.style-venv\Scripts\clang-format.exe --dry-run --Werror src\student_report.cpp
```

Linux:

```
.style-venv/bin/clang-format --dry-run --Werror src/student_report.cpp
```

macOS:

```
.style-venv/bin/clang-format --dry-run --Werror src/student_report.cpp
```

Обычно при успехе команда ничего не печатает. Сообщение `code should be clang-formatted` означает, что требуется форматирование. Это не ошибка алгоритма и не сообщение компилятора.

5.3. Анализировать один .cpp

PowerShell:

```
.style-venv\Scripts\clang-tidy.exe src\student_report.cpp -- -std=c++17 -Iinclude
```

CMD:

```
.style-venv\Scripts\clang-tidy.exe src\student_report.cpp -- -std=c++17 -Iinclude
```

Linux:

```
.style-venv/bin/clang-tidy src/student_report.cpp -- -std=c++17 -Iinclude
```

macOS:

```
.style-venv/bin/clang-tidy src/student_report.cpp -- \
    -std=c++17 -Iinclude -stdlib=libc++ \
    -isysroot "$(xcrun --show-sdk-path)" \
    -isystem "$(dirname "$(xcrun --find clang++)")/../include/c++/v1"
```

До `--` стоят параметры анализатора, после — параметры разбора C++. `-std=c++17` задает стандарт; `-Iinclude` указывает каталог собственных заголовков. На macOS дополнительно задаются SDK и заголовки libc++ из установленного инструментария Apple; готовый скрипт получает эти пути через xcrun. Для всех файлов удобнее запускать check_style.py: заголовки проверяются через подключающие их .cpp.

5.4. Проверить файл настроек

PowerShell:

```
.style-venv\Scripts\clang-tidy.exe --verify-config
```

CMD:

```
.style-venv\Scripts\clang-tidy.exe --verify-config
```

Linux:

```
.style-venv/bin/clang-tidy --verify-config
```

macOS:

```
.style-venv/bin/clang-tidy --verify-config
```

Успешный результат — `No config errors detected.`. Неизвестная проверка или неизвестное имя параметра обычно означает опечатку либо несовместимую версию. Для .clang-format проверкой является успешный запуск форматирования с этим файлом; неизвестная настройка не должна игнорироваться.

5.5. Как узнать результат команды

Успех соответствует коду завершения 0; другое значение означает ошибку. Сразу после нужной команды выполните блок своей оболочки:

PowerShell:

```
$LASTEXITCODE
```

CMD:

```
echo %errorlevel%
```

Linux:

```
echo $?
```

macOS:

```
echo $?
```

Смотрите также текст диагностики. Проверки нельзя считать пройденными, если запуск остановился из-за отсутствующих заголовков или неверных параметров сборки.

6. Как прочитать сообщение clang-tidy

Пример типовой диагностики:

```
src/example.cpp:12:9: error: invalid case style for function 'calculateAverage'
[readability-identifier-naming]
```

`src/example.cpp` — файл, 12 — строка, 9 — столбец. Текст объясняет проблему. Название в квадратных скобках — правило, которое сработало. Здесь функцию следует назвать

CalculateAverage и обновить ее использования средствами переименования редактора. Если это обязательное имя внешнего API, его не меняют: исключение рассматривается для конкретного интерфейса, а не отключает все проверки имен.

`modernize-use-override` означает, что у переопределения отсутствует override. `readability-braces-around-statements` означает, что у условия или цикла нет фигурных скобок. `clang-diagnostic-error` часто означает, что код не разобран из-за настоящей ошибки C++ или неполной конфигурации компиляции. Сначала исправляются такие ошибки, затем остальные сообщения.

Автоматическое `--fix` не входит в обычный процесс курса: для переименований важно проверить все места использования, а для остальных изменений понять результат. Не добавляйте NOLINT-комментарии: соглашение курса запрещает комментарии в собственном коде. Обоснованные исключения согласуются отдельно и ограничиваются нужным API или конкретной проверкой.

7. Что означают настройки комплекта

7.1. .clang-format

`BasedOnStyle: Microsoft` выбирает основу. `IndentWidth: 4` задает размер отступа. `UseTab: Never` исключает табуляцию. `BreakBeforeBraces: Allman` переносит фигурную скобку. `AccessModifierOffset: -4` выводит public/protected/private на уровень class. `NamespaceIndentation: None` не добавляет отступ внутри пространства имен.

`ColumnLimit: 100` задает целевую ширину строки. `PointerAlignment: Left` и `DerivePointerAlignment: false` закрепляют запись `Type* pointer` и `Type& reference`. Параметры AllowShort... не позволяют сворачивать функции, условия, циклы и перечисления в короткие строки. `SortIncludes: CaseSensitive` сортирует подключения внутри групп. `FixNamespaceComments: false` не добавляет комментарий после namespace; имеющиеся комментарии этот параметр не удаляет.

7.2. .clang-tidy

`Checks` начинается с `-*`, то есть выключает общий набор, затем включает перечисленные правила. Проверка имен задает PascalCase для типов и методов, camelCase для переменных, `m_` для закрытых полей, `s_` для изменяемых статических полей и `c_` для соответствующих констант. В терминах clang-tidy PascalCase называется CamelCase, а camelCase — camelBack. Main и обязательные имена внешних переопределений не переименовываются.

Помимо имен включены скобки у условий и циклов, раздельные объявления переменных, override, nullptr, использование `= default` для подходящих специальных операций, инициализация полей, корректная модель деструктора полиморфного класса, некоторые случаи использования объекта после move и лишнего копирования в range-for. Список намеренно ограничен: он дополняет требования курса, а не автоматически проверяет весь язык C++.

`HeaderFilterRegex: '.*'` позволяет показывать сообщения для подключенных несистемных заголовков. В крупном проекте со сторонними библиотеками фильтр следует сузить до собственных каталогов. Чужой код не должен переписываться ради проверки курса.

Порядок членов класса, отсутствие комментариев, смысл имен и правильность алгоритма готовая пара инструментов полностью не проверяет. Эти пункты оцениваются отдельно. Новые настройки в существующий файл вносятся с сохранением YAML-отступов: табуляция и произвольные вложенности недопустимы.

8. Сборка рабочего примера

На Windows нужен терминал разработчика Visual Studio. Первая команда создает каталог сборки, если его нет. Итоговая программа печатает `Average: 4`.

PowerShell:

```
New-Item -ItemType Directory -Force build | Out-Null
cl /nologo /std:c++17 /EHsc /W4 /WX /permissive- /utf-8 /Iinclude `
    src\main.cpp src\report.cpp src\student_report.cpp `
    /Fo:build\ /Fe:build\course_style_example.exe
.\build\course_style_example.exe
```

CMD:

```
if not exist build mkdir build
cl /nologo /std:c++17 /EHsc /W4 /WX /permissive- /utf-8 /Iinclude ^
    src\main.cpp src\report.cpp src\student_report.cpp ^
    /Fo:build\ /Fe:build\course_style_example.exe
build\course_style_example.exe
```

Linux:

```
mkdir -p build
g++ -std=c++17 -Wall -Wextra -Wpedantic -Wshadow -Werror -Iinclude \
    src/main.cpp src/report.cpp src/student_report.cpp \
    -o build/course_style_example
./build/course_style_example
```

macOS:

```
mkdir -p build
clang++ -std=c++17 -Wall -Wextra -Wpedantic -Wshadow -Werror -Iinclude \
    src/main.cpp src/report.cpp src/student_report.cpp \
    -o build/course_style_example
./build/course_style_example
```

Запускайте итоговую программу только если компиляция завершилась успешно. При неуспешной сборке в каталоге может остаться старый исполняемый файл; его запуск не подтверждает исправление кода.

9. Подключение редактора

9.1. Visual Studio Code

Установите расширение C/C++ от Microsoft и откройте папку проекта. В настройках рабочей папки укажите C/C++ Formatting = clangFormat, Clang format style = file и путь к установленному clang-format. Форматирование документа доступно через Format Document; при желании включите Format On Save. Встроенная в расширение версия может отличаться от закрепленной, поэтому путь указывается явно.

Фрагмент `.vscode/settings.json` для Windows, когда окружение уже создано:

```
{
    "C_Cpp.formatting": "clangFormat",
    "C_Cpp.clang_format_style": "file",
    "C_Cpp.clang_format_path": "${workspaceFolder}/.style-venv/Scripts/clang-format.exe",
    "[cpp]": {
        "editor.defaultFormatter": "ms-vscode.cpptools",
        "editor.formatOnSave": true
    }
}
```

Для Linux и macOS меняется только путь: `${workspaceFolder}/.style-venv/bin/clang-format`. Если в settings.json уже есть настройки, добавьте поля в существующий объект, сохранив правильные запятые, а не вставляйте второй отдельный JSON-объект. Команда check_style.py остается контрольным запуском независимо от редактора.

9.2. Visual Studio и другие среды

Современная Visual Studio поддерживает clang-format, но настройки и встроенная версия зависят от выпуска. Если встроенное форматирование расходится с проверкой, примените готовый скрипт перед сдачей. В другой среде ищите подключение clang-format из файла и укажите исполняемый файл из `.style-venv`. Подключение редактора необязательно для работы комплекта.

10. Когда проект станет сложнее: compile_commands.json

Для этого примера достаточно `-- -std=c++17 -Iinclude`. В проекте с дополнительными библиотеками, defines и разными параметрами файлов нужен compile_commands.json: база фактических команд компиляции. Тогда clang-tidy получает параметры из базы, а не из приблизительно составленной строки.

При установленных CMake и Ninja база создается генератором Ninja. На Windows снова используется терминал разработчика. Каталог `build-tidy` должен быть отдельным: уже настроенный каталог другого генератора не переиспользуется. Здесь CMake и Ninja — дополнительный вариант; их установка не нужна для обычных команд выше.

PowerShell:

```
cmake -S . -B build-tidy -G Ninja `
    -DCMAKE_BUILD_TYPE=Release -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
cmake --build build-tidy
.style-venv\Scripts\python.exe check_style.py --build-dir build-tidy
```

CMD:

```
cmake -S . -B build-tidy -G Ninja ^
    -DCMAKE_BUILD_TYPE=Release -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
cmake --build build-tidy
.style-venv\Scripts\python.exe check_style.py --build-dir build-tidy
```

Linux:

```
cmake -S . -B build-tidy -G Ninja \
    -DCMAKE_BUILD_TYPE=Release -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
cmake --build build-tidy
.style-venv/bin/python check_style.py --build-dir build-tidy
```

macOS:

```
cmake -S . -B build-tidy -G Ninja \
    -DCMAKE_BUILD_TYPE=Release -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
cmake --build build-tidy
.style-venv/bin/python check_style.py --build-dir build-tidy
```

Генераторы CMake Visual Studio не создают эту базу с помощью CMAKE_EXPORT_COMPILE_COMMANDS; подходят Ninja и Makefile-генераторы. Для ручного анализа применяется `clang-tidy путь/к/файлу.cpp -p build-tidy`; параметры после `--` при этом обычно не нужны. Перед анализом проверьте, что база существует и сборка настроена для нужной системы.

11. Частые проблемы

Команда не найдена

Проверьте текущую папку, наличие `.style-venv` и используйте полный относительный путь из инструкции. После установки Python откройте новый терминал. На Windows `py` относится к запускателю Python; на Linux и macOS исходная команда — python3.

File not found: cstddef, vector или iostream

Это отсутствие стандартных заголовков или их путей, а не ошибка стиля. На Windows проверьте компонент C++ и откройте терминал разработчика. На Linux установите компилятор и заголовки стандартной библиотеки. На macOS проверьте инструменты Apple и пути SDK. Для сложного проекта используйте базу команд компиляции. Не создавайте собственный пустой файл с именем отсутствующего стандартного заголовка.

Не найден report.h или другой собственный заголовок

Выполните команду из корня проекта и проверьте `-Iinclude`. Имя файла и регистр букв должны совпадать с include: Windows может скрыть ошибку регистра, которая проявится в Linux.

Форматирование выглядит как Google или LLVM

Проверьте, что исходник расположен внутри папки с .clang-format, нет скрытого .txt и вложенной конфигурации. В редакторе выберите форматирование по файлу и нужную версию программы. Если приложение показывает несохраненный буфер, сначала сохраните файл: командный инструмент читает содержимое с диска.

Появляются неизвестные настройки

Проверьте версии инструментов и целостность файла. Используйте комплект вместе с tools-requirements.txt. Не удаляйте неизвестный параметр наугад только для получения зеленой проверки.

После форматирования остаются ошибки имен

Это ожидаемо: clang-format не переименовывает. Исправьте имена по диагностике clang-tidy, включая объявления и вызовы, затем повторите проверку.

Программа проверку прошла, но считает неправильно

Стиль и выбранный статический анализ не доказывают корректность результата. Нужны сборка, запуск и проверки входных данных. Особенно проверьте пустой набор, граничные значения и ошибочный ввод.

12. Что отправлять преподавателю

Отправляются собственные include, src, файлы настроек, необходимые файлы сборки и отчет по заданию. `.style-venv`, build, служебные папки IDE и исполняемые файлы не отправляются, если условие не требует их отдельно. Перед отправкой выполняются форматирование, проверка, успешная сборка и проверка поведения.

13. Официальные справочные материалы

• LLVM — ClangFormat: запуск форматирования и поиск конфигурации.

• LLVM — Clang-Tidy: параметры анализа и база компиляции.

• Python — venv: локальное окружение без обязательной активации.

• Python — Downloads: установщики Python.

• PyPI — clang-format: пакет программы форматирования.

• PyPI — clang-tidy: пакет анализатора.

• Microsoft — C++ Build Tools: компоненты C++ и терминалы разработчика.

• Microsoft — WinGet install: установка пакетов Windows.

• Homebrew — Installation: установка Homebrew в macOS.

• VS Code — C++ formatting: настройка редактора.

• CMake — CMAKE_EXPORT_COMPILE_COMMANDS: поддерживаемые генераторы базы.

Дата подготовки: 12 октября 2026 года. Конфигурации, рабочий пример и команды анализа и сборки проверены на Windows с MSVC. Инструкции Linux и macOS составлены по документации; запуск на этих ОС в рамках подготовки комплекта не выполнялся.